

MCA Semester 1	Subject : Advanced Data Structures Lab
Name : Mukund Gangurde	Topic: MaxHeap
Roll No. : MCA2511	Date : 19-12-2025

Code:

```
import java.util.Scanner;
```

```
class MaxHeap
```

```
{
    int[] arr;          //Array to store the heap elements
    int cnt; //Current number of elements in the heap
    int n;              //maximum capacity of the heap - size of array

    //Constructor
    public MaxHeap(int size)
    {
        n = size;
        arr = new int[n];
        cnt = 0;
    }

    //Get parent index
    public int parent(int i)
    {
        return ((i-1)/2);
    }

    //Get Left Child
    public int leftChild(int i)
    {
        return ((2*i)+1);
    }

    //Get Right Child
    public int rightChild(int i)
    {
        return ((2*i)+2);
    }

    //Insert an element in the top
    public void Insert(int x)
    {
        if(cnt==n)
```

```
{
    System.out.println("Heap is Full!..");
    return;
}

//Insert at the end of the Array
arr[cnt] = x;
int curr = cnt;
cnt++;

//reheapUp to maintain order property
while(curr>=0 && arr[curr]>arr[parent(curr)])
{
    swap(curr,parent(curr));
    curr = parent(curr);
}
} //end of insert

//Swap
public void swap(int i, int j)
{
    int tmp = arr[i];
    arr[i] = arr[j];
    arr[j] = tmp;
}

//Display
public void Display()
{
    System.out.print("Heap: ");

    for(int i=0; i<cnt; i++)
    {
        System.out.print(arr[i] + " ");
    }
}

//Delete - return the element at the root - largest element
public void Delete()
{
    if(cnt==0)
    {
        System.out.println("Heap is empty!....");
        return;
    }
}
```

```
int tmp = arr[0];           //Hold largest value in tmp
arr[0] = arr[cnt-1];        //Replace root with last leaf node -- almost complete
cnt--;                      //Reduce count of elements
reheapDown(0);              //reheapDown from root
System.out.println("Element deleted: "+tmp);
}

//reheapDown to maintain order property of the heap
public void reheapDown(int i)
{
    int max = i;
    int left = leftChild(i);
    int right = rightChild(i);

    //Find the maximum of root, left and right child
    if(left<cnt && arr[left] > arr[max])
    {
        max = left;
    }
    if(right<cnt && arr[right] > arr[max])
    {
        max = right;
    }

    //If max is not at root, reheapDown and continue
    if(max!=i)
    {
        swap(i,max);
        reheapDown(max);
    }
}

} //end of MaxHeap

//MAIN
class Heap
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        MaxHeap h = new MaxHeap(10);
        int ch;

        do
        {
            System.out.println("\n-----Max-Heap-----\n");
```

```
System.out.println("1. Insert in the Heap");
System.out.println("2. Delete from the Heap");
System.out.println("3. Display the Heap");
System.out.println("4. Exit\n");

System.out.print("Enter your choice: ");
ch = sc.nextInt();

switch(ch)
{
    case 1:
        System.out.print("Enter a value: ");
        int x = sc.nextInt();
        h.Insert(x);
        break;

    case 2:
        h.Delete();
        break;

    case 3:
        h.Display();
        break;

    case 4:
        System.out.println("Exiting....");
        break;

    default:
        System.out.println("Incorrect Choice ");
        break;
}
} while(ch!=4);
}
```

Output:

```
A:\MCA2511\DS_LAB>javac 23Heap.java
```

```
A:\MCA2511\DS_LAB>java Heap
```

```
-----Max-Heap-----
```

1. Insert in the Heap
2. Delete from the Heap
3. Display the Heap
4. Exit

Enter your choice: 1

Enter a value: 50

```
-----Max-Heap-----
```

1. Insert in the Heap
2. Delete from the Heap
3. Display the Heap
4. Exit

Enter your choice: 1

Enter a value: 30

```
-----Max-Heap-----
```

1. Insert in the Heap
2. Delete from the Heap
3. Display the Heap
4. Exit

Enter your choice: 1

Enter a value: 20

```
-----Max-Heap-----
```

1. Insert in the Heap
2. Delete from the Heap
3. Display the Heap
4. Exit

Enter your choice: 1

Enter a value: 55

```
-----Max-Heap-----
```

1. Insert in the Heap
2. Delete from the Heap
3. Display the Heap
4. Exit

Enter your choice: 1

Enter a value: 45

```
-----Max-Heap-----
```

1. Insert in the Heap
2. Delete from the Heap
3. Display the Heap
4. Exit

Enter your choice: 1

Enter a value: 80

-----Max-Heap-----

1. Insert in the Heap
2. Delete from the Heap
3. Display the Heap
4. Exit

Enter your choice: 1
Enter a value: 15

-----Max-Heap-----

1. Insert in the Heap
2. Delete from the Heap
3. Display the Heap
4. Exit

Enter your choice: 1
Enter a value: 8

-----Max-Heap-----

1. Insert in the Heap
2. Delete from the Heap
3. Display the Heap
4. Exit

Enter your choice: 1
Enter a value: 70

-----Max-Heap-----

1. Insert in the Heap
2. Delete from the Heap
3. Display the Heap
4. Exit

Enter your choice: 3
Heap: 80 70 55 50 45 20 15 8 30

-----Max-Heap-----

1. Insert in the Heap
2. Delete from the Heap
3. Display the Heap
4. Exit

Enter your choice: 2
Element deleted: 80

-----Max-Heap-----

1. Insert in the Heap
2. Delete from the Heap
3. Display the Heap
4. Exit

Enter your choice: 3
Heap: 70 50 55 30 45 20 15 8

-----Max-Heap-----

1. Insert in the Heap
2. Delete from the Heap
3. Display the Heap
4. Exit

Enter your choice: 2
Element deleted: 70

-----Max-Heap-----

1. Insert in the Heap
2. Delete from the Heap
3. Display the Heap
4. Exit

Enter your choice: 3
Heap: 55 50 20 30 45 8 15

-----Max-Heap-----

1. Insert in the Heap
2. Delete from the Heap
3. Display the Heap
4. Exit

Enter your choice: 2
Element deleted: 55

-----Max-Heap-----

1. Insert in the Heap
2. Delete from the Heap
3. Display the Heap
4. Exit

Enter your choice: 3
Heap: 50 45 20 30 15 8

-----Max-Heap-----

1. Insert in the Heap
2. Delete from the Heap
3. Display the Heap
4. Exit

Enter your choice: 2
Element deleted: 50

-----Max-Heap-----

1. Insert in the Heap
2. Delete from the Heap
3. Display the Heap
4. Exit

Enter your choice: 2
Element deleted: 45

-----Max-Heap-----

1. Insert in the Heap
2. Delete from the Heap
3. Display the Heap
4. Exit

Enter your choice: 2
Element deleted: 30

-----Max-Heap-----

1. Insert in the Heap
2. Delete from the Heap
3. Display the Heap
4. Exit

Enter your choice: 3
Heap: 20 15 8

-----Max-Heap-----

1. Insert in the Heap
2. Delete from the Heap
3. Display the Heap
4. Exit

Enter your choice: 2
Element deleted: 20

-----Max-Heap-----

1. Insert in the Heap
2. Delete from the Heap
3. Display the Heap
4. Exit

Enter your choice: 2
Heap is empty!....

-----Max-Heap-----

1. Insert in the Heap
2. Delete from the Heap
3. Display the Heap
4. Exit

Enter your choice: 2
Element deleted: 15

-----Max-Heap-----

1. Insert in the Heap
2. Delete from the Heap
3. Display the Heap
4. Exit

Enter your choice: 2
Element deleted: 8

-----Max-Heap-----

1. Insert in the Heap
2. Delete from the Heap
3. Display the Heap
4. Exit

Enter your choice: 3
Heap: